

Exercice récapitulatif

Gestion d'une médiathèque

Définition

Un CRUD est l'ensemble des quatre opérations de base que toute application doit savoir faire sur une collection de données :

Lettre	Opération	Question posée	Signature typique
C	Create	« J'ajoute un élément »	<code>boolean ajouter(T element)</code>
R	Read	« Je consulte / je recherche »	<code>T rechercherParId(int id)</code> <code>List<T> listerTous()</code>
U	Update	« Je modifie un élément existant »	<code>boolean modifier(int id, ...)</code>
D	Delete	« Je supprime un élément »	<code>boolean supprimer(int id)</code>

Contexte

La médiathèque « Le Signet » veut remplacer son cahier papier.

Elle gère :

- un catalogue de médias : des livres, des DVD et des jeux vidéo ;
- des membres inscrits ;
- des emprunts : qui a emprunté quoi, depuis quand, et pour combien de temps encore.

Vous allez écrire l'application console qui pilote tout ça.

Architecture imposée

mediatheque/

|—— Main.java

point d'entrée, HORS des autres packages

|—— models/

| |—— Media.java

(abstract)

| |—— Livre.java

(extends Media)

| |—— Dvd.java

(extends Media)

| |—— JeuVideo.java

(extends Media)

| |—— Membre.java

| |—— Emprunt.java

|—— enums/

| |—— Genre.java

| |—— Plateforme.java

|—— services/

| |—— Mediatheque.java

le cœur : CRUD + gestion

|—— utils/

| |—— ConsoleUtils.java

(méthodes static uniquement)

|—— configs/

| |—— Config.java

(constantes static final uniquement)

Cette découpe n'est pas cosmétique : elle sépare ce que sont les choses (models) de ce qu'on en fait (services), et c'est exactement ce que vous retrouverez dans n'importe quel projet professionnel.

Partie A : Le modèle

A.1 - Les énumérations

```
public enum Genre {  
    ACTION, AVENTURE, POLICIER, SCIENCE_FICTION, DOCUMENTAIRE, JEUNESSE  
}  
  
public enum Plateforme { PS5, XBOX, SWITCH, PC }
```

A.2 - La classe abstraite Media

Attribut	Type	Contrainte
id	int	attribué automatiquement, jamais modifiable après création
titre	String	jamais null ni vide
anneeSortie	int	entre 1900 et l'année courante
genre	Genre	jamais null
disponible	boolean	true à la création ; aucun setter public

Points d'attention :

1. L'id s'auto-incrémente via un attribut `private static int nextId = 1;`
le même mécanisme que NB_PERSONNES de la démo 09, mais utilisé pour identifier au lieu de compter.
2. Deux méthodes abstraites :
 - `public abstract int dureeEmpruntJours();`
→ 21 pour un livre, 7 pour un DVD, 14 pour un jeu.
 - `public abstract String typeLibelle();`
→ "Livre", "DVD", "Jeu vidéo".
3. La disponibilité ne change que via deux méthodes au visibilité restreinte au package `models` : `void marquerEmprunte()` et `void marquerDisponible()`. Un code extérieur ne doit pas pouvoir rendre un média disponible sans passer par la médiathèque.
4. Redéfinissez `toString()`, `equals()` et `hashCode()` (basés sur l'id seul).
5. `getId()` doit être final : aucune sous-classe ne peut mentir sur son identité.

A.3 - Les trois sous-classes

Classe	Attributs supplémentaires	Durée d'emprunt
Livre	auteur (String), nbPages (int > 0), isbn (String)	21 jours
Dvd	realisateur (String), dureeMinutes (int > 0)	7 jours
JeuVideo	plateforme (Plateforme), pegi (int {3, 7, 12, 16, 18}, ou enum)	14 jours

Chacune : constructeur qui appelle `super(...)`, `toString()` qui appelle `super.toString()`, `equals()/hashCode()` qui appellent `super.equals()/super.hashCode()`.

A.4 - Membre

id (auto-incrémenté), nom, prenom, email, dateInscription (LocalDate).

`equals()/hashCode()` sur email (deux personnes homonymes existent, deux adresses identiques non).

A.5 - Emprunt

Attribut	Type	Contraintes
media	Media	
membre	Membre	
dateEmprunt	LocalDate	
dateRetourPrevue	LocalDate	
dateRetourReelle	LocalDate	null tant que le média n'est pas rendu

Le constructeur reçoit un Media et un Membre, fixe `dateEmprunt` à aujourd'hui et calcule `dateRetourPrevue` tout seul, à partir de `media.dureeEmpruntJours()`. C'est du polymorphisme : Emprunt ne sait pas, et n'a pas à savoir, s'il manipule un livre ou un DVD.

Méthodes :

- `boolean estEnCours()` → `dateRetourReelle == null`
- `boolean estEnRetard()` → en cours et date prévue dépassée
- `long joursDeRetard()` → 0 si pas de retard (`ChronoUnit.DAYS.between(...)`)

- `long cloturer()` → renseigne `dateRetourReelle` à aujourd'hui, libère le média et renvoie le nombre de jours de retard

Partie B : Le CRUD

Tout se passe dans `services/Mediatheque.java`

```
public class Mediatheque {

    private final String nom;
    private final Map<Integer, Media> catalogue = new HashMap<>();
    private final Map<Integer, Membre> membres = new HashMap<>();
    private final List<Emprunt> emprunts = new ArrayList<>();

    // ...
}
```

B.1 - CREATE

Méthode	Comportement attendu
<code>boolean ajouterMedia(Media media)</code>	refuse null, refuse un média déjà présent (<code>containsKey</code>), renvoie true si l'ajout a eu lieu
<code>void ajouterMedias(Media... medias)</code>	varargs - délègue à la méthode ci-dessus pour chaque élément, sans dupliquer la moindre ligne de logique
<code>boolean inscrireMembre(Membre membre)</code>	refuse null et refuse un email déjà inscrit

B.2 - READ

Méthode	Comportement attendu
<code>Media rechercherMediaParId(int id)</code>	renvoie le média ou null
<code>List<Media> listerTous()</code>	renvoie une copie de la collection <code>new ArrayList<>(catalogue.values())</code>
<code>List<Media> rechercher(String motCle)</code>	tous les médias dont le titre contient le mot-clé, insensible à la casse
<code>List<Media> rechercher(Genre genre)</code>	surcharge : filtre par genre
<code>List<Media> listerDisponibles()</code>	uniquement ceux qui ne sont pas empruntés
<code>List<Media> listerParType(Class<?> type) (bonus)</code>	instanceof / isInstance

`listerTous()` renvoie une copie et pas `catalogue.values()` directement. Pourquoi ?

Testez les deux versions : dans un cas, un appelant malintentionné peut faire

`mediatheque.listerTous().clear()` et vider votre catalogue depuis l'extérieur.

C'est le prolongement direct de la démo 06 : encapsuler, c'est aussi protéger ce qu'on renvoie.

B.3 - UPDATE

```
public boolean modifierMedia(  
    int id, String nouveauTitre, int nouvelleAnnee, Genre nouveauGenre) { ... }
```

- Renvoie false si l'ID n'existe pas (ne crée jamais un média au passage).
- Passe par les setters du modèle, qui valident. La médiathèque ne réécrit pas les règles de validation : elle fait confiance au modèle. Si vous devez recopier ici les contrôles écrits dans Media, c'est que vos setters sont trop laxistes.

B.4 - DELETE

```
public boolean supprimerMedia(int id)
```

Deux gardes, chacun avec un return false immédiat (comme `reserver()` de l'exercice cinéma) :

1. l'id n'existe pas ;
2. le média est actuellement emprunté → suppression interdite ;

Le chemin nominal se lit ensuite d'un seul trait, sans aucun if imbriqué.

Partie C : La gestion

C.1 - Emprunter

```
public boolean emprunter(int idMedia, int idMembre)
```

Gardes, dans cet ordre :

1. le média n'existe pas ;
2. le membre n'existe pas ;
3. le média n'est pas disponible ;
4. le membre a déjà `Config.MAX_EMPRUNTS_PAR_MEMBRE` emprunts en cours.

Chemin nominal : créer l'Emprunt, l'ajouter à la liste, appeler `media.marquerEmprunte()`.

C.2 - Rendre

```
public double rendre(int idMedia)
```

Renseigne `dateRetourReelle`, libère le média.

C.3 - Consulter et statistiques

Méthode	Rôle
<code>List<Emprunt> empruntsEnCours()</code>	tous les emprunts non rendus
<code>List<Emprunt> empruntsEnCours(Membre membre)</code>	surcharge : ceux d'un membre donné
<code>List<Emprunt> listerRetards()</code>	tous les emprunts en retard
<code>void afficherStatistiques()</code>	voir le format ci-dessous

Statistiques attendues :

```
=== Statistiques - Le Signet ===
Médias au catalogue : 12 (7 livres, 3 DVD, 2 jeux video)
Disponibles          : 9
Empruntés            : 3
Membres inscrits     : 4
Genre le plus représenté : POLICIER (5 médias)
```

Le comptage par type se fait avec `instanceof` dans une seule boucle (revoyez `compterEspeces()` de `l'animalerie`).

Le « genre le plus représenté » demande une `Map<Genre, Integer>` : c'est l'exercice le plus formateur du lot.

C.4 - Config et ConsoleUtils

configs/Config.java

→ que des constantes, aucun attribut d'instance, aucun constructeur public :

```
public class Config {
    public static final int MAX_EMPRUNTS_PAR_MEMBRE = 3;
    public static final String NOM_MEDIATHEQUE = "Le Signet";

    private Config() {} // on n'instancie pas une classe utilitaire
}
```

utils/ConsoleUtils.java

→ que des méthodes static :

- `lireEntier(String message)` :
- `lireTexte(String message)` :
- `lireOuiNon(String message)` :
- `afficherTitre(String titre)` :
- `afficherListe(List<?> elements)` :

Elles encapsulent le Scanner et le protègent des saisies farfelues.

Partie D : L'interface console

Dans **Main.java**, une boucle de menu :

```
=====
MÉDIATHÈQUE LE SIGNET
=====
1. Ajouter un média
2. Lister le catalogue
3. Rechercher un média
4. Modifier un média
5. Supprimer un média
-----
6. Inscrire un membre
7. Emprunter
8. Rendre
9. Emprunts en cours
-----
10. Statistiques
0. Quitter
=====
Votre choix :
```

Contraintes :

- une méthode static par option, le main ne contient que la boucle et le switch ;
- le switch utilise la syntaxe fléchée (`case 1 -> ajouterMedia(mediatheque);`) ;
- aucun `System.out.println` dans Mediatheque : le service calcule et renvoie, le Main affiche. C'est la règle la plus importante de tout l'exercice ;
- le programme démarre avec un jeu de données pré-chargé (une méthode `chargerDonneesDemo()`) pour ne pas devoir tout ressaisir à chaque test.

Exemple de session

```
Votre choix : 7
ID du media : 3
ID du membre : 1
OK - "Blade Runner" emprunte par Dupont Alice. Retour prévu le 2026-08-24.

Votre choix : 7
ID du media : 3
ID du membre : 2
REFUS - Ce media est deja emprunte.

Votre choix : 8
ID du media : 3
Retour enregistre. Retard : 9 jours.
```

Partie E : Auto-contrôle

Restreindre trop provoque une erreur de compilation : le problème se voit tout de suite.

Restreindre trop peu ne provoque rien. D'où cette vérification, obligatoire, exactement comme dans l'exercice du cinéma.

Depuis Main, ces lignes doivent refuser de compiler :

```
media.disponible = true;           // attribut private
media.marquerDisponible();         // visible seulement dans le package models
mediatheque.catalogue.clear();    // collection private
new Media("titre", 2020, Genre.ACTION); // classe abstraite
```

Et ce test doit passer sans laisser de trace :

```
mediatheque.listerTous().clear();
System.out.println(mediatheque.listerTous().size()); // doit toujours afficher 12
```

Enfin, ce test valide equals() / hashCode() :

```
Set<Media> set = new HashSet<>();
set.add(livre);
set.add(livre);
System.out.println(set.size()); // doit afficher 1, pas 2
```

Paliers

Palier	Contenu	Pour qui
Niveau 1 - le socle	Parties A et B (modèle + CRUD complet), menu réduit aux options 1 à 5	tout le monde, c'est le minimum évalué
Niveau 2 - la gestion	Partie C entière + menu complet	l'objectif normal de fin de module

Documentation officielle

- Tutoriel Java - Collections :
<https://docs.oracle.com/javase/tutorial/collections/>
- API java.util.Map :
<https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/Map.html>
- API java.util.List :
<https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/List.html>
- API java.time.LocalDate :
<https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/time/LocalDate.html>
- Contrat equals() / hashCode() :
[https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/Object.html#equals
\(java.lang.Object\)](https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/Object.html#equals(java.lang.Object))
- Tutoriel Java - Classes abstraites :
<https://docs.oracle.com/javase/tutorial/java/land/abstract.html>